The Diophantine equation $x^2 = n! \cdot n\#$: the complete solution set, formalized and machine-checked in Lean 4

Aurora0134*

October 6, 2026

Abstract

We determine the complete solution set of the Diophantine equation $x^2 = n! \cdot n\#$ (the factorial times the primorial): for $n \in \mathbb{N}$ the equation is solvable exactly for $n \in \{0, 1, 2, 3, 4, 5\}$, with $x = 1, 1, 2, 6, 12, 60$. This settles the first of the two cases that Novaković (2026) recorded as believed-finite with the words "at the moment we were not able to give an argument". The proof combines a valuation criterion ($n! \cdot n\#$ is a square if and only if $\nu_p(n!)$ is odd for every prime $p \leq n$), an interval-prime lemma (for $n \geq 22$ there is a prime in $(n/3, n/2]$) proved by a prime-certificate chain up to 10^6 and an Erdős-style factorial-ratio estimate beyond it, and explicit witnesses for small n. Every statement is machine-checked in Lean 4 against mathlib v4.34.0: the statements were frozen by hash before proving, the proof compiles with no `sorry`, and `#print axioms` reports only {propext, `Quot.sound`, `Classical.choice`}. The results were developed with AI assistance under human direction, and the Lean 4 kernel is the sole adjudicator. Lean sources, frozen statements, and audit logs accompany the paper as a deposited artifact, and a companion claim of record is deposited separately.

1 Introduction

The equation of the title asks when the product of the factorial and the primorial is a perfect square. It appears, together with the companion equation $x^2 = n!! \cdot [1, \ldots, n]$, in Novaković's 2026 study [3] of the Diophantine equation

*Contact: via the GitHub account Aurora0134.

$P(x) = \prod_{i=1}^{r} H_{n_i}$, where the author writes (verbatim wording; mathematical notation retypeset): "We believe that finitely many solutions occur for $x^2 = n! \cdot n\#$ or $x^2 = n!! \cdot [1, \ldots, n]$, but at the moment we were not able to give an argument". The present paper answers the first of these two cases completely: the equation $x^2 = n! \cdot n\#$ has, for $n \in \mathbb{N}$, exactly the solutions $n \in \{0, 1, 2, 3, 4, 5\}$. The second case is not claimed here.

Three contributions are recorded together. First, the mathematical result itself, as an if-and-only-if characterization with explicit witnesses (Section 4); an exclusivity review found no occupying record for this statement on the channels it could reach (Section 7). Second, a complete machine-checked formalization: eight statements frozen by hash before proving, proved in Lean 4 against mathlib with no placeholder and with axiom dependencies exactly the three standard ones (Section 6). Third, the artifact is built so that a reader can re-run the adjudication — the frozen statement's hash, the strict compile's exit status and the axiom printout all ship with the deposit — rather than take the text's word for it. This is why the paper addresses a formal-methods readership as well as a number-theoretic one.

A companion claim of record for the same kernel result was deposited separately [11]; the two deposits are byte-level siblings at the level of the proof sources and differ in genre: the claim note is a priority record, this paper is the narrative account.

Notation

We write $n!$ for the factorial and $n\# = \prod_{p \le n,\ p \text{ prime}} p$ for the primorial; $\nu_p(m)$ denotes the exponent of the prime p in m. In the formalization, `primorial` is the top-level mathlib declaration and $\nu_p(m)$ is `Nat.factorization`. Division in the interval-prime lemma (Lemma 5) is natural-number division, so the interval reads $\lfloor n/3 \rfloor < p \le \lfloor n/2 \rfloor$; for prime existence this coincides with the real reading, since p is an integer.

2 Mathematical background

Definition 1. For $n \in \mathbb{N}$, the *primorial* $n\#$ is the product of the primes $p \le n$ (so $0\# = 1\# = 1$, $5\# = 30$). For a prime p and $m \ge 1$, $\nu_p(m)$ is the exponent of p in the prime factorization of m.

The only classical ingredient the proof uses beyond arithmetic is Legendre's formula $\nu_p(n!) = \sum_{i \ge 1} \lfloor n/p^i \rfloor$, which enters through mathlib's `Nat.factorization_factorial`. The shape of the argument is: a square has

all valuations even (Lemma 4 reduces the equation to a parity condition on $\nu_p(n!)$); a prime p with $2p \le n < 3p$ contributes $\nu_p(n!) = \lfloor n/p \rfloor = 2$, an even number, provided $p^2 > n$ kills the higher terms; and such a prime exists for all $n \ge 22$ (Lemma 5). The small cases are decided pointwise.

Where this sits in the literature. The equation belongs to the family around the Brocard–Ramanujan equation $n! + 1 = m^2$, of which Novaković's equation $P(x) = \prod_{i=1}^{r} H_{n_i}$ [3] is a generalization; the same author's companion paper [4] is recorded here with the restricted wording that its object of study does not contain $n\#$. Adjacent objects and records that must be distinguished from the present statement — the squarefree part of $n!$ (OEIS A055204 [9]), the disproved Erdős problem #399 [8], primorial-core Pell equations [5], square products of factorials [6], and the interval-prime literature (Nagura [7], OEIS A166968 [10]) — are delimited one by one in Section 7.

3 Formalisation setup

The formalization is written against Lean 4 [1], version `v4.34.0` (the `leanprover/lean4` release of that version), with mathlib4 [2] `v4.34.0` at revision `5ed29652`; the pin ships with the artifact (`proofs/lean-toolchain`). The eight statements were frozen by hash before proving began; the proving stage may replace proof bodies only, and the final audit re-compares the delivered statement headers against the frozen snapshot under two whitespace conventions.

Encoding decisions. Four encoding choices are worth recording. (i) The primorial is mathlib's top-level `primorial`; the primorial valuations enter through the squarefreeness of `primorial` and the divisibility characterization `Nat.Prime.dvd_primorial_iff` (`primorial_factorization_eq_one`, `primorial_factorization_eq_zero`). (ii) The main result is delivered in two forms: the iff form `isSquare_factorial_mul_primorial_iff` ("$n! \cdot n\#$ is a square if and only if $n \le 5$") and the existence-witness form `exists_sq_factorial_mul_primorial_iff` ("$x^2 = n! \cdot n\#$ is solvable if and only if $n \in \{0, 1, 2, 3, 4, 5\}$"), bridged by `isSquare_iff_exists_sq`; on $\mathbb{N}$, $n \le 5$ and $n \in \{0, \ldots, 5\}$ coincide. (iii) The interval-prime lemma is stated with natural-number division; this is faithful to the real reading for the purpose of prime existence, as recorded in the formalization's fidelity note. (iv) The criterion of Lemma 4 carries the hypothesis $1 \le n$ as frozen;

the proof does not use it (the criterion holds at $n = 0$ as well), and the lemma is stated as frozen rather than strengthened — Section 8(e).

Audit mechanism. The statement is frozen by hash; a strict compile must exit 0 with no placeholder (`sorry`/`admit`) anywhere in the file, comments included; `#print axioms` must report only the three standard axioms; and the statement comparison is re-run by an independent audit at the end. The verdicts for this paper are reproduced in Section 6 and ship with the artifact.

4 Main results

Theorem 2 (complete solution set, iff form)**.** *For every $n \in \mathbb{N}$, $n! \cdot n\#$ is a perfect square if and only if $n \leq 5$.*

Theorem 3 (existence-witness form)**.** *For every $n \in \mathbb{N}$, the equation $x^2 = n! \cdot n\#$ has a solution $x \in \mathbb{N}$ if and only if $n \in \{0, 1, 2, 3, 4, 5\}$.*

Lemma 4 (valuation criterion)**.** *For $n \geq 1$: $n! \cdot n\#$ is a perfect square if and only if $\nu_p(n!)$ is odd for every prime $p \leq n$.*

Lemma 5 (interval-prime lemma)**.** *For every integer $n \geq 22$ there exists a prime p with $n/3 < p \leq n/2$.*

Theorem 6 (solution witnesses)**.** *$1^2 = 0! \cdot 0\#$, $1^2 = 1! \cdot 1\#$, $2^2 = 2! \cdot 2\#$, $6^2 = 3! \cdot 3\#$, $12^2 = 4! \cdot 4\#$, and $60^2 = 5! \cdot 5\#$.*

Three auxiliary factorization statements belong to the same frozen set and carry the same grade (verbatim in Listing 1): $\nu_p(n\#) = 1$ for primes $p \leq n$, $\nu_p(n\#) = 0$ for primes $p \nleq n$, and $\nu_p(n! \cdot n\#) = \nu_p(n!) + 1$ for primes $p \leq n$.

Listing 1: Frozen statement snapshot (Lean 4): the complete frozen set, copied from the `lean` fence of the snapshot shipped as `proofs/nfact-primorial-square.statement.txt` (a comment-cleaned edition: docstrings translated to English for publication; the statement text is byte-identical to the frozen snapshot of record, sha256 `0b13d08d`; correspondence record `audit/comment-cleanup-diff.md`).

```
/-- **Main theorem (complete solution set, iff form)**:
    `n! * n#` is a perfect square ⟺ `n ≤ 5` (for `n : ℕ`).
    Over ℕ, `n ≤ 5 ⟺ n ∈ {0,1,2,3,4,5}`, so this is the main
    statement "the solutions are exactly n ∈ {0,...,5}". -/
theorem isSquare_factorial_mul_primorial_iff (n : ℕ) :
```

```
    IsSquare (n ! * primorial n) ↔ n ≤ 5

/-- **Main theorem (existence-witness form, delivered alongside
    the iff form)**:
    the Diophantine equation `x^2 = n! * n#` has a solution ⟺ `n
    ∈ {0,1,2,3,4,5}`. -/
theorem exists_sq_factorial_mul_primorial_iff (n : ℕ) :
    (∃ x : ℕ, x ^ 2 = n ! * primorial n) ↔ n ∈ ({0, 1, 2, 3, 4,
    5} : Finset ℕ)

/-- **Lemma A (equivalent valuation criterion)**: for `n ≥ 1`, `n
    ! * n#` is a perfect square ⟺
    `ν_p(n!)` is odd for every prime `p ≤ n`. -/
theorem isSquare_iff_all_factorization_odd (n : ℕ) (hn : 1 ≤ n) :
    IsSquare (n ! * primorial n) ↔
      ∀ p : ℕ, p.Prime → p ≤ n → Odd ((n !).factorization p)

/-- **Lemma B (interval prime lemma)**: for every integer `n ≥
    22` there exists a prime `p` with `n/3 < p ≤ n/2`. -/
theorem exists_prime_between_div_three_and_div_two (n : ℕ) (hn :
    22 ≤ n) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2

/-- **Witness layer (explicit square roots for the ⟸ direction
    of the main theorem)**: `x^2 = n! * n#` at each of n = 0..5,
    with x = 1, 1, 2, 6, 12, 60. -/
theorem witnesses_zero_to_five :
    1 ^ 2 = 0 ! * primorial 0 ∧
    1 ^ 2 = 1 ! * primorial 1 ∧
    2 ^ 2 = 2 ! * primorial 2 ∧
    6 ^ 2 = 3 ! * primorial 3 ∧
    12 ^ 2 = 4 ! * primorial 4 ∧
    60 ^ 2 = 5 ! * primorial 5

/-- Auxiliary statement: `ν_p(n#) = 1` (primes `p ≤ n`). -/
theorem primorial_factorization_eq_one {p n : ℕ} (hp : p.Prime) (
    hpn : p ≤ n) :
    (primorial n).factorization p = 1

/-- Auxiliary statement: `ν_p(n#) = 0` (primes `p ≰ n`). -/
theorem primorial_factorization_eq_zero {p n : ℕ} (hp : p.Prime) (
    hpn : ¬ p ≤ n) :
    (primorial n).factorization p = 0

/-- Auxiliary statement (the multiplicative skeleton of the
```

```
    criterion): `ν_p(n! * n#) = ν_p(n!) + 1` (primes `p ≤ n`). -/
40 theorem factorization_factorial_mul_primorial {p n : ℕ} (hp : p.
    Prime) (hpn : p ≤ n) :
41     (n ! * primorial n).factorization p = (n !).factorization p +
    1
```

5 Proofs

The final proof file (1094 lines) is a single Lean module in two namespaces, and the proof assembles four components: the valuation criterion, the exclusion of large n via the interval-prime lemma, the small-n witness layer, and the assembly of the two main theorems. The file ships byte-exactly in the artifact under `proofs/`; the excerpts below are byte-identical contiguous blocks of it (independent mechanical record: `audit/listing-verify.txt`, which ships in the deposit). The shipped file is the comment-cleaned edition (English docstrings); its code is byte-identical to the frozen development (record: `audit/comment-cleanup-diff.md`). We describe the proof in the order in which it is built.

5.1 The valuation criterion

The multiplicative spine of the argument is `factorization_factorial_mul_primorial` : for a prime $p \le n$, the primorial contributes p exactly once (it is squarefree and divisible by p), so

$$\nu_p(n! \cdot n\#) = \nu_p(n!) + 1.$$

A natural number is a square if and only if all its prime valuations are even (mathlib's `Nat.isSquare_iff_even_factorization`). For $p \le n$ the displayed identity makes the evenness of $\nu_p(n! \cdot n\#)$ equivalent to the oddness of $\nu_p(n!)$; for $p > n$ the prime divides neither $n!$ nor $n\#$, so the valuation is 0, an even number. This proves Lemma 4 in both directions. The backward direction is discharged through the library bridge rather than by an explicit square-root construction — a design-stage rerouting (evidence by kernel prechecks), not a weakening.

5.2 The witness layer

The six solutions are exhibited pointwise (Theorem 6, closed by decision). For the remaining small cases, the range $6 \le n \le 21$ is excluded by sixteen

explicit factorizations: each point exhibits a prime p with $\nu_p(n!\cdot n\#) = 2e+1$ odd — for instance $\nu_2(6!\cdot 6\#) = 5$ since $6!\cdot 6\# = 2^5\cdot 675$ with $2\nmid 675$ — and the bridge `not_isSquare_of_odd_factorization` turns an odd valuation into a non-square (a square $m = r\cdot r$ would have $\nu_p(m) = \nu_p(r) + \nu_p(r)$ even). The assembly `nonsquare_six_to_twenty_one` closes the range by case analysis. The sixteen witnesses appear in full in Listing 4.

5.3 The interval-prime lemma: the certificate chain

For $22 \le n \le 999999$ the lemma is closed by a certificate chain (Listing 2). The single hop `lemmaB_hop` observes that a prime q with $2q \le n \le 3q-1$ witnesses $n/3 < q \le n/2$. A greedy chain of twenty-nine primes,

$$11, 13, 19, 23, 31, 43, 61, 89, 131, 193, 283, 421, 631, 941, 1409, 2113, 3169, 4751, 7121, 10667, \ldots, 409609,$$

together with twenty-eight continuity certificates $2q' \le 3q$ between consecutive primes, covers the segment from $22 = 2\cdot 11$ up to $3\cdot 409609 - 1 = 1228826 \ge 999999$; every certificate is a one-line `norm_num`/`omega` fact, and the chain's decision tree is `lemmaB_upto_999999`. The chain was produced by a mechanical generator and kernel-checked as a whole; the segment $22 \le n \le 32$ is covered by the first hop with the uniform witness $p = 11$.

Listing 2: Excerpt of the proof file (lines 96–168 of `final-nfact-primorial-square.lean`, which ships in full in `proofs/`): the certificate-chain mechanism for Lemma 5 on $22 \le n \le 999999$ — the two boundary certificates, the single-hop lemma `lemmaB_hop`, and the twenty-nine-hop decision tree.

```
theorem chain_start : 22 = 2 * 11 := by omega
theorem chain_end : 999999 ≤ 3 * 409609 - 1 := by omega

/-- Certificate-chain single hop: for prime `q` and `n ∈ [2q, 3q - 1]`,
    `q` itself witnesses `n/3 < q ≤ n/2`.
    Note: extract `hq.two_le` first, otherwise `omega` gets stuck at the
    `q = 0` counterexample under saturating subtraction on ℕ. -/
theorem lemmaB_hop {q : ℕ} (hq : q.Prime) {n : ℕ} (h1 : 2 * q ≤ n) (h2 :
     n ≤ 3 * q - 1) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 :=
  have hq2 : 2 ≤ q := hq.two_le
  ⟨q, hq, by omega, by omega⟩

/-- **Lemma B, finite segment (certificate-chain decision tree)**: for
    every `n ∈ [22, 999999]`,
    `∃ p, p.Prime ∧ n/3 < p ∧ p ≤ n/2`.
```

```
    Joins the tail theorem (which starts at n = 1000000) by a case split
    on `n < 1000000`. -/
theorem lemmaB_upto_999999 (n : ℕ) (h1 : 22 ≤ n) (h2 : n ≤ 999999) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 := by
  rcases lt_or_ge n 33 with h | h
  · exact lemmaB_hop (q := 11) prime_cert_11 (by omega) (by omega)
  · rcases lt_or_ge n 39 with h | h
    · exact lemmaB_hop (q := 13) prime_cert_13 (by omega) (by omega)
    · rcases lt_or_ge n 57 with h | h
      · exact lemmaB_hop (q := 19) prime_cert_19 (by omega) (by omega)
      · rcases lt_or_ge n 69 with h | h
        · exact lemmaB_hop (q := 23) prime_cert_23 (by omega) (by omega)
        · rcases lt_or_ge n 93 with h | h
          · exact lemmaB_hop (q := 31) prime_cert_31 (by omega) (by omega)
          · rcases lt_or_ge n 129 with h | h
            · exact lemmaB_hop (q := 43) prime_cert_43 (by omega) (by
    omega)
            · rcases lt_or_ge n 183 with h | h
              · exact lemmaB_hop (q := 61) prime_cert_61 (by omega) (by
    omega)
              · rcases lt_or_ge n 267 with h | h
                · exact lemmaB_hop (q := 89) prime_cert_89 (by omega) (by
    omega)
                · rcases lt_or_ge n 393 with h | h
                  · exact lemmaB_hop (q := 131) prime_cert_131 (by omega)
    (by omega)
                  · rcases lt_or_ge n 579 with h | h
                    · exact lemmaB_hop (q := 193) prime_cert_193 (by omega
    ) (by omega)
                    · rcases lt_or_ge n 849 with h | h
                      · exact lemmaB_hop (q := 283) prime_cert_283 (by
    omega) (by omega)
                      · rcases lt_or_ge n 1263 with h | h
                        · exact lemmaB_hop (q := 421) prime_cert_421 (by
    omega) (by omega)
                        · rcases lt_or_ge n 1893 with h | h
                          · exact lemmaB_hop (q := 631) prime_cert_631 (by
     omega) (by omega)
                          · rcases lt_or_ge n 2823 with h | h
                            · exact lemmaB_hop (q := 941) prime_cert_941 (
    by omega) (by omega)
                            · rcases lt_or_ge n 4227 with h | h
                              · exact lemmaB_hop (q := 1409)
    prime_cert_1409 (by omega) (by omega)
                              · rcases lt_or_ge n 6339 with h | h
                                · exact lemmaB_hop (q := 2113)
    prime_cert_2113 (by omega) (by omega)
                                · rcases lt_or_ge n 9507 with h | h
                                  · exact lemmaB_hop (q := 3169)
```

```
    prime_cert_3169 (by omega) (by omega)
51                                  · rcases lt_or_ge n 14253 with h | h
52                                    · exact lemmaB_hop (q := 4751)
    prime_cert_4751 (by omega) (by omega)
53                                    · rcases lt_or_ge n 21363 with h | h
54                                      · exact lemmaB_hop (q := 7121)
    prime_cert_7121 (by omega) (by omega)
55                                      · rcases lt_or_ge n 32001 with h | h
56                                        · exact lemmaB_hop (q := 10667)
    prime_cert_10667 (by omega) (by omega)
57                                        · rcases lt_or_ge n 47973 with h |
     h
58                                          · exact lemmaB_hop (q := 15991)
    prime_cert_15991 (by omega) (by omega)
59                                          · rcases lt_or_ge n 71943 with h
     | h
60                                            · exact lemmaB_hop (q :=
    23981) prime_cert_23981 (by omega) (by omega)
61                                            · rcases lt_or_ge n 107907
    with h | h
62                                              · exact lemmaB_hop (q :=
    35969) prime_cert_35969 (by omega) (by omega)
63                                              · rcases lt_or_ge n 161853
    with h | h
64                                                · exact lemmaB_hop (q :=
    53951) prime_cert_53951 (by omega) (by omega)
65                                                · rcases lt_or_ge n 242769
     with h | h
66                                                  · exact lemmaB_hop (q :=
     80923) prime_cert_80923 (by omega) (by omega)
67                                                  · rcases lt_or_ge n
    364137 with h | h
68                                                    · exact lemmaB_hop (q
    := 121379) prime_cert_121379 (by omega) (by omega)
69                                                    · rcases lt_or_ge n
    546177 with h | h
70                                                      · exact lemmaB_hop (
    q := 182059) prime_cert_182059 (by omega) (by omega)
71                                                      · rcases lt_or_ge n
    819249 with h | h
72                                                        · exact lemmaB_hop
     (q := 273083) prime_cert_273083 (by omega) (by omega)
73                                                        · exact lemmaB_hop
     (q := 409609) prime_cert_409609 (by omega) (by omega)
```

5.4 The interval-prime lemma: the factorial-ratio tail

For $n \geq 10^6$ the lemma is closed by an Erdős-style factorial-ratio argument (Listing 3). With

$$\mathrm{tailR}(n) = \frac{(n/2)!\,(n/6)!}{(n/3)!\,(n/4)!\,(n/12)!}$$

(natural-number division throughout), the valuations are $\nu_p(\mathrm{tailR}(n)) = \sum_{i\geq 1} \mathrm{hval}(\lfloor n/p^i \rfloor)$, where $\mathrm{hval}(m) = \lfloor m/2 \rfloor + \lfloor m/6 \rfloor - \lfloor m/3 \rfloor - \lfloor m/4 \rfloor - \lfloor m/12 \rfloor$ takes only the values 0 and 1, with $\mathrm{hval}(m) = 1$ exactly when $m \bmod 12 \in \{2, 6, 7, 8, 10, 11\}$. Two bounds collide. On the lower side, an induction on blocks of twelve (`tailR_lower_twelve`, `tailR_lower_strict`) gives the strict estimate

$$18^{\lfloor n/12 \rfloor} < \mathrm{tailR}(n) \cdot (4\lfloor n/12 \rfloor + 3)^6.$$

On the upper side, if the interval $(n/3, n/2]$ contained no prime, then the positions with hval $= 1$ in the critical range would be excluded, and the total mass collapses to $\mathrm{tailR}(n) \leq (n/2)^{\sqrt{n/2}} \cdot 4^{n/6}$ (`tailR_le_of_no_prime`), with $4^{n/6} \leq 4 \cdot 16^{\lfloor n/12 \rfloor}$. The decisive closure

$$4 \cdot (4q+3)^6 \cdot (6q+5)^{\sqrt{6q+5}} \cdot 16^q \leq 18^q \qquad (q \geq 10^5)$$

(`tail_main_inequality_nat`) is a real-variable concavity two-point estimate (`concave_log_affine`, `real_tail_inequality`), modelled on mathlib's Bertrand proof; the margin is the bare inequality $18 > 16$. The join at the boundary, $10^6 \leq n < 1.2 \cdot 10^6$, is a single certificate hop with $q = 400009$.

Listing 3: Excerpt of the proof file (lines 735–767): the tail theorem for $n \geq 10^6$ (`lemmaB_tail_1000000`), closing Lemma 5 on the infinite regime via the factorial-ratio estimate.

```
theorem lemmaB_tail_1000000 (n : ℕ) (hn : 1000000 ≤ n) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 := by
  rcases lt_or_ge n 1200000 with h | h
  · exact lemmaB_hop (q := 400009) (by norm_num) (by omega) (by omega)
  · by_contra hc
    push_neg at hc
    have hc' : ∀ p : ℕ, p.Prime → n / 3 < p → ¬ p ≤ n / 2 := by
      intro p hp1 hp2 hle
      have hcc := hc p hp1 hp2
      omega
    have hub := tailR_le_of_no_prime n (by omega) hc'
    have hlb := tailR_lower_strict n hn
```

```
    have hmain := tail_main_inequality_nat (n/12) (by omega)
    have h2 : n < (n/12 + 1) * 12 := (Nat.div_lt_iff_lt_mul (by norm_num
    : (0:ℕ) < 12)).mp (Nat.lt_succ_self _)
    have e1 : (n/2)^(Nat.sqrt (n/2)) ≤ (6*(n/12)+5)^(Nat.sqrt (6*(n/12)
    +5)) := by
      have hbase : n/2 ≤ 6*(n/12)+5 := by omega
      have hs : Nat.sqrt (n/2) ≤ Nat.sqrt (6*(n/12)+5) := Nat.
    sqrt_le_sqrt hbase
      exact (Nat.pow_le_pow_right (by omega : 0 < n/2) hs).trans (Nat.
    pow_le_pow_left hbase _)
    have e2 : 4^(n/6) ≤ 4 * 16^(n/12) := by
      have h4 : n/6 ≤ 2*(n/12) + 1 := by omega
      calc 4^(n/6) ≤ 4^(2*(n/12)+1) := Nat.pow_le_pow_right (by norm_num)
     h4
        _ = 4 * 16^(n/12) := by
          rw [pow_succ, pow_mul]
          norm_num
          ring
    have key : 18^(n/12) < 18^(n/12) := by
      calc 18^(n/12) < tailR n * (4*(n/12)+3)^6 := hlb
        _ ≤ ((n/2)^(Nat.sqrt (n/2)) * 4^(n/6)) * (4*(n/12)+3)^6 := Nat.
    mul_le_mul_right _ hub
        _ ≤ ((6*(n/12)+5)^(Nat.sqrt (6*(n/12)+5)) * (4 * 16^(n/12))) *
    (4*(n/12)+3)^6 :=
            Nat.mul_le_mul_right _ (Nat.mul_le_mul e1 e2)
        _ = 4 * (4*(n/12)+3)^6 * (6*(n/12)+5)^(Nat.sqrt (6*(n/12)+5)) *
    16^(n/12) := by ring
        _ ≤ 18^(n/12) := hmain
    exact absurd key (lt_irrefl _)
```

5.5 Assembly of the main theorems

The backward direction of Theorem 2 is the witness layer: for $n \leq 5$, case analysis (`interval_cases`) produces the explicit square roots $x = 1, 1, 2, 6, 12, 60$ via `isSquare_iff_exists_sq`. For the forward direction, suppose $n! \cdot n\#$ is a square and $n \geq 6$. If $6 \leq n \leq 21$, the witness layer contradicts it directly. If $n \geq 22$, Lemma 5 yields a prime p with $n/3 < p \leq n/2$; then $2p \leq n < 3p$ and $p^2 \geq 3p > n$ (using $p \geq 3$), so Legendre's formula collapses to its first term:

$$\nu_p(n!) = \sum_{i \geq 1} \lfloor n/p^i \rfloor = \lfloor n/p \rfloor = 2$$

(in the formalization, `Nat.factorization_factorial` is applied with the bound $b = 2$, since $p^2 > n$ gives `Nat.log` $p\, n < 2$ via `Nat.log_lt_of_lt_pow`). But the criterion of Lemma 4 forces $\nu_p(n!)$ to be odd — contradiction.

Theorem 3 is then the bridge through `isSquare_iff_exists_sq` together with the finite-set reading of $n \leq 5$ on $\mathbb{N}$.

Listing 4: Excerpt of the proof file (lines 771–1094): the main namespace `NFactPrimorialSquare` in full — the three auxiliary factorization lemmas, Lemma 4, the witness layer, the sixteen non-square witnesses, Lemma 5 assembled from the chain and the tail, and the two main theorems.

```
namespace NFactPrimorialSquare

/-- Auxiliary statement: `ν_p(n#) = 1` for primes `p ≤ n`. -/
theorem primorial_factorization_eq_one {p n : ℕ} (hp : p.Prime) (hpn : p
    ≤ n) :
    (primorial n).factorization p = 1 :=
  factorization_eq_one_of_squarefree (squarefree_primorial n) hp
    ((Nat.Prime.dvd_primorial_iff hp).mpr hpn)

/-- Auxiliary statement: `ν_p(n#) = 0` for primes `p ≰ n`. -/
theorem primorial_factorization_eq_zero {p n : ℕ} (hp : p.Prime) (hpn : ¬
     p ≤ n) :
    (primorial n).factorization p = 0 :=
  Nat.factorization_eq_zero_of_not_dvd
    (fun h => hpn ((Nat.Prime.dvd_primorial_iff hp).mp h))

/-- Auxiliary statement (the multiplicative skeleton of the criterion): `
    ν_p(n! * n#) = ν_p(n!) + 1` for primes `p ≤ n`. -/
theorem factorization_factorial_mul_primorial {p n : ℕ} (hp : p.Prime) (
    hpn : p ≤ n) :
    (n ! * primorial n).factorization p = (n !).factorization p + 1 := by
  rw [Nat.factorization_mul (Nat.factorial_ne_zero n) (primorial_ne_zero
    n),
      Finsupp.add_apply, primorial_factorization_eq_one hp hpn]

/-- **Lemma A (equivalent valuation criterion)**: for `n ≥ 1`, `n! * n#`
    is a perfect square ⟺
    `ν_p(n!)` is odd for every prime `p ≤ n`.
    (`Nat.factorization m p` is ν_p(m): the exponent of the prime `p` in
    `m`.) -/
theorem isSquare_iff_all_factorization_odd (n : ℕ) (hn : 1 ≤ n) :
    IsSquare (n ! * primorial n) ↔
      ∀ p : ℕ, p.Prime → p ≤ n → Odd ((n !).factorization p) := by
  constructor
  · intro h p hp hpn
    have hEven : Even ((n ! * primorial n).factorization p) :=
      Nat.isSquare_iff_even_factorization.mp h p hp
    rw [factorization_factorial_mul_primorial hp hpn] at hEven
    obtain ⟨m, hm⟩ := hEven
    cases m with
```

```
    | zero => simp at hm
    | succ k => exact ⟨k, by omega⟩
  · intro h
    apply Nat.isSquare_iff_even_factorization.mpr
    intro p hp
    by_cases hpn : p ≤ n
    · rw [factorization_factorial_mul_primorial hp hpn]
      exact (h p hp hpn).add_one
    · rw [Nat.factorization_mul (Nat.factorial_ne_zero n) (
    primorial_ne_zero n),
        Finsupp.add_apply,
        Nat.factorization_eq_zero_of_not_dvd
          (fun hd => hpn ((Nat.Prime.dvd_factorial hp).mp hd)),
        primorial_factorization_eq_zero hp hpn]
      exact ⟨0, rfl⟩

/-- **Witness layer (explicit square roots for the ⟸ direction of the
    main theorem)**: `x^2 = n! * n#`
    at each of n = 0..5, with x = 1, 1, 2, 6, 12, 60 (the pointwise
    correspondence of the main statement). -/
theorem witnesses_zero_to_five :
    1 ^ 2 = 0 ! * primorial 0 ∧
    1 ^ 2 = 1 ! * primorial 1 ∧
    2 ^ 2 = 2 ! * primorial 2 ∧
    6 ^ 2 = 3 ! * primorial 3 ∧
    12 ^ 2 = 4 ! * primorial 4 ∧
    60 ^ 2 = 5 ! * primorial 5 := by decide

/-- Bridge lemma (internal to the witness layer): an odd factorization at
     some prime implies non-square.
    `IsSquare` gives `m = r * r`, so `ν_p(m) = ν_p(r) + ν_p(r)` is even,
    contradicting the oddness assumption. -/
theorem not_isSquare_of_odd_factorization {m p e : ℕ} (hm : m ≠ 0)
    (h : m.factorization p = 2 * e + 1) : ¬ IsSquare m := by
  rintro ⟨r, hr⟩
  have hne : r ≠ 0 := by rintro rfl; rw [mul_zero] at hr; exact hm hr
  rw [hr, Nat.factorization_mul hne hne, Finsupp.add_apply] at h
  omega

/-- Non-square witness n = 6: `ν_2(6! * 6#) = 5` (odd); `6! * 6# = 2^5 *
    675`, and `2 ∤ 675`. -/
theorem nonsquare_at_6 : ¬ IsSquare (6 ! * primorial 6) := by
  have hM : (6 : ℕ) ! * primorial 6 = 2 ^ 5 * 675 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 2)
    (mul_ne_zero (Nat.factorial_ne_zero 6) (primorial_ne_zero 6))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^5) ≠ 0) (by
    norm_num : (675:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 675),
```

```
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 7: `ν_2(7! * 7#) = 5` (odd); `7! * 7# = 2^5 *
    33075`, and `2 ∤ 33075`. -/
theorem nonsquare_at_7 : ¬ IsSquare (7 ! * primorial 7) := by
  have hM : (7 : ℕ) ! * primorial 7 = 2 ^ 5 * 33075 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 2)
    (mul_ne_zero (Nat.factorial_ne_zero 7) (primorial_ne_zero 7))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^5) ≠ 0) (by
    norm_num : (33075:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 33075),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 8: `ν_3(8! * 8#) = 3` (odd); `8! * 8# = 3^3 *
    313600`, and `3 ∤ 313600`. -/
theorem nonsquare_at_8 : ¬ IsSquare (8 ! * primorial 8) := by
  have hM : (8 : ℕ) ! * primorial 8 = 3 ^ 3 * 313600 := by decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 1)
    (mul_ne_zero (Nat.factorial_ne_zero 8) (primorial_ne_zero 8))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^3) ≠ 0) (by
    norm_num : (313600:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 313600),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 9: `ν_3(9! * 9#) = 5` (odd); `9! * 9# = 3^5 *
    313600`, and `3 ∤ 313600`. -/
theorem nonsquare_at_9 : ¬ IsSquare (9 ! * primorial 9) := by
  have hM : (9 : ℕ) ! * primorial 9 = 3 ^ 5 * 313600 := by decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 2)
    (mul_ne_zero (Nat.factorial_ne_zero 9) (primorial_ne_zero 9))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^5) ≠ 0) (by
    norm_num : (313600:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 313600),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 10: `ν_2(10! * 10#) = 9` (odd); `10! * 10# =
    2^9 * 1488375`, and `2 ∤ 1488375`. -/
theorem nonsquare_at_10 : ¬ IsSquare (10 ! * primorial 10) := by
```

```
  have hM : (10 : ℕ) ! * primorial 10 = 2 ^ 9 * 1488375 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 4)
    (mul_ne_zero (Nat.factorial_ne_zero 10) (primorial_ne_zero 10))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^9) ≠ 0) (by
    norm_num : (1488375:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 1488375),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 11: `ν_2(11! * 11#) = 9` (odd); `11! * 11# =
    2^9 * 180093375`, and `2 ∤ 180093375`. -/
theorem nonsquare_at_11 : ¬ IsSquare (11 ! * primorial 11) := by
  have hM : (11 : ℕ) ! * primorial 11 = 2 ^ 9 * 180093375 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 4)
    (mul_ne_zero (Nat.factorial_ne_zero 11) (primorial_ne_zero 11))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^9) ≠ 0) (by
    norm_num : (180093375:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 180093375),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 12: `ν_2(12! * 12#) = 11` (odd); `12! * 12# =
    2^11 * 540280125`, and `2 ∤ 540280125`. -/
theorem nonsquare_at_12 : ¬ IsSquare (12 ! * primorial 12) := by
  have hM : (12 : ℕ) ! * primorial 12 = 2 ^ 11 * 540280125 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 5)
    (mul_ne_zero (Nat.factorial_ne_zero 12) (primorial_ne_zero 12))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^11) ≠ 0) (by
    norm_num : (540280125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 540280125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 13: `ν_2(13! * 13#) = 11` (odd); `13! * 13# =
    2^11 * 91307341125`, and `2 ∤ 91307341125`. -/
theorem nonsquare_at_13 : ¬ IsSquare (13 ! * primorial 13) := by
  have hM : (13 : ℕ) ! * primorial 13 = 2 ^ 11 * 91307341125 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 5)
    (mul_ne_zero (Nat.factorial_ne_zero 13) (primorial_ne_zero 13))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^11) ≠ 0) (by
    norm_num : (91307341125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 91307341125),
```

```
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 14: `ν_5(14! * 14#) = 3` (odd); `14! * 14# =
    5^3 * 20943712677888`, and `5 ∤ 20943712677888`. -/
theorem nonsquare_at_14 : ¬ IsSquare (14 ! * primorial 14) := by
  have hM : (14 : ℕ) ! * primorial 14 = 5 ^ 3 * 20943712677888 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 5) (e := 1)
    (mul_ne_zero (Nat.factorial_ne_zero 14) (primorial_ne_zero 14))
  rw [hM, Nat.factorization_mul (by norm_num : ((5:ℕ)^3) ≠ 0) (by
    norm_num : (20943712677888:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (5:ℕ) ∣ 20943712677888),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization (by norm_num : Nat.Prime 5), Finsupp.
    single_eq_same]; norm_num [nsmul_eq_mul]

/-- Non-square witness n = 15: `ν_3(15! * 15#) = 7` (odd); `15! * 15# =
    3^7 * 17955857920000`, and `3 ∤ 17955857920000`. -/
theorem nonsquare_at_15 : ¬ IsSquare (15 ! * primorial 15) := by
  have hM : (15 : ℕ) ! * primorial 15 = 3 ^ 7 * 17955857920000 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 3)
    (mul_ne_zero (Nat.factorial_ne_zero 15) (primorial_ne_zero 15))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^7) ≠ 0) (by
    norm_num : (17955857920000:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 17955857920000),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 16: `ν_3(16! * 16#) = 7` (odd); `16! * 16# =
    3^7 * 287293726720000`, and `3 ∤ 287293726720000`. -/
theorem nonsquare_at_16 : ¬ IsSquare (16 ! * primorial 16) := by
  have hM : (16 : ℕ) ! * primorial 16 = 3 ^ 7 * 287293726720000 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 3)
    (mul_ne_zero (Nat.factorial_ne_zero 16) (primorial_ne_zero 16))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^7) ≠ 0) (by
    norm_num : (287293726720000:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 287293726720000),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

```

```
/-- Non-square witness n = 17: `ν_3(17! * 17#) = 7` (odd); `17! * 17# =
    3^7 * 83027887022080000`, and `3 ∤ 83027887022080000`. -/
theorem nonsquare_at_17 : ¬ IsSquare (17 ! * primorial 17) := by
  have hM : (17 : ℕ) ! * primorial 17 = 3 ^ 7 * 83027887022080000 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 3)
    (mul_ne_zero (Nat.factorial_ne_zero 17) (primorial_ne_zero 17))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^7) ≠ 0) (by
    norm_num : (83027887022080000:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 83027887022080000),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 18: `ν_2(18! * 18#) = 17` (odd); `18! * 18# =
    2^17 * 24936491397943125`, and `2 ∤ 24936491397943125`. -/
theorem nonsquare_at_18 : ¬ IsSquare (18 ! * primorial 18) := by
  have hM : (18 : ℕ) ! * primorial 18 = 2 ^ 17 * 24936491397943125 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 8)
    (mul_ne_zero (Nat.factorial_ne_zero 18) (primorial_ne_zero 18))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^17) ≠ 0) (by
    norm_num : (24936491397943125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 24936491397943125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 19: `ν_2(19! * 19#) = 17` (odd); `19! * 19# =
    2^17 * 9002073394657468125`, and `2 ∤ 9002073394657468125`. -/
theorem nonsquare_at_19 : ¬ IsSquare (19 ! * primorial 19) := by
  have hM : (19 : ℕ) ! * primorial 19 = 2 ^ 17 * 9002073394657468125 :=
    by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 8)
    (mul_ne_zero (Nat.factorial_ne_zero 19) (primorial_ne_zero 19))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^17) ≠ 0) (by
    norm_num : (9002073394657468125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 9002073394657468125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 20: `ν_2(20! * 20#) = 19` (odd); `20! * 20# =
    2^19 * 45010366973287340625`, and `2 ∤ 45010366973287340625`. -/
theorem nonsquare_at_20 : ¬ IsSquare (20 ! * primorial 20) := by
  have hM : (20 : ℕ) ! * primorial 20 = 2 ^ 19 * 45010366973287340625 :=
```

```
    by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 9)
    (mul_ne_zero (Nat.factorial_ne_zero 20) (primorial_ne_zero 20))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^19) ≠ 0) (by
    norm_num : (45010366973287340625:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 45010366973287340625),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 21: `ν_2(21! * 21#) = 19` (odd); `21! * 21# =
    2^19 * 945217706439034153125`, and `2 ∤ 945217706439034153125`. -/
theorem nonsquare_at_21 : ¬ IsSquare (21 ! * primorial 21) := by
  have hM : (21 : ℕ) ! * primorial 21 = 2 ^ 19 * 945217706439034153125 :=
     by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 9)
    (mul_ne_zero (Nat.factorial_ne_zero 21) (primorial_ne_zero 21))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^19) ≠ 0) (by
    norm_num : (945217706439034153125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 945217706439034153125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Assembled form (cited by the ⟹ direction of the main theorem): `n!
    * n#` is non-square at every `n` with `6 ≤ n ≤ 21`. -/
theorem nonsquare_six_to_twenty_one (n : ℕ) (h6 : 6 ≤ n) (h21 : n ≤ 21)
    :
    ¬ IsSquare (n ! * primorial n) := by
  interval_cases n
  · exact nonsquare_at_6
  · exact nonsquare_at_7
  · exact nonsquare_at_8
  · exact nonsquare_at_9
  · exact nonsquare_at_10
  · exact nonsquare_at_11
  · exact nonsquare_at_12
  · exact nonsquare_at_13
  · exact nonsquare_at_14
  · exact nonsquare_at_15
  · exact nonsquare_at_16
  · exact nonsquare_at_17
  · exact nonsquare_at_18
  · exact nonsquare_at_19
  · exact nonsquare_at_20
  · exact nonsquare_at_21

```

```
/-- **Lemma B (interval prime lemma)**: for every integer `n ≥ 22` there exists a prime `p` with `n/3 < p ≤ n/2`.
    (`/` is natural-number division: the interval reads `⌊n/3⌋ < p ≤ ⌊n/2⌋`; for prime existence this coincides
    with the real reading, since `p` is an integer: `n/3 < p ⟺ ⌊n/3⌋ < p` and `p ≤ n/2 ⟺ p ≤ ⌊n/2⌋`.) -/
theorem exists_prime_between_div_three_and_div_two (n : ℕ) (hn : 22 ≤ n) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 := by
  rcases lt_or_ge n 1000000 with h | h
  · exact NFactPrimorialSquareChain.lemmaB_upto_999999 n hn (by omega)
  · exact NFactPrimorialSquareChain.lemmaB_tail_1000000 n h

/-- **Main theorem (complete solution set, iff form)**:
    `n! * n#` is a perfect square ⟺ `n ≤ 5` (for `n : ℕ`).
    Over ℕ, `n ≤ 5 ⟺ n ∈ {0,1,2,3,4,5}`, so this is the main
    statement "the solutions are exactly n ∈ {0,...,5}". -/
theorem isSquare_factorial_mul_primorial_iff (n : ℕ) :
    IsSquare (n ! * primorial n) ↔ n ≤ 5 := by
  constructor
  · -- ⟹ direction: n! * n# a square ⟹ n ≤ 5 (contrapositive: for n ≥ 6, n! * n# is not a square)
    intro h
    by_contra hle
    have h6 : 6 ≤ n := by omega
    rcases lt_or_ge n 22 with hlt | h22
    · -- the 6 ≤ n ≤ 21 segment: direct contradiction with the pointwise non-square witnesses of the witness layer
      exact nonsquare_six_to_twenty_one n h6 (by omega) h
    · -- the 22 ≤ n segment: Lemma B yields a prime p ∈ (n/3, n/2]; the criterion gives Odd (ν_p(n!)), but Legendre's sum evaluates ν_p(n!) = 2
      obtain ⟨p, hp, hpl, hpu⟩ := exists_prime_between_div_three_and_div_two n h22
      have hodd : Odd ((n !).factorization p) :=
        (isSquare_iff_all_factorization_odd n (by omega)).mp h p hp (by omega)
      have hn3p : n < 3 * p := by omega
      have hp3 : 3 ≤ p := by omega
      have hpsq : n < p ^ 2 := by
        have h3pp : 3 * p ≤ p * p := Nat.mul_le_mul hp3 (le_refl p)
        rw [pow_two]
        exact lt_of_lt_of_le hn3p h3pp
      have hlog2 : Nat.log p n < 2 := Nat.log_lt_of_lt_pow (by omega) hpsq
      have hd1 : n / p = 2 := by
        have hp0 : 0 < p := hp.pos
        have hr : n - 2 * p < p := by omega
        rw [show n = n - 2 * p + p * 2 by omega, Nat.add_mul_div_left _ _
```

```
      hp0,
286         Nat.div_eq_of_lt hr, zero_add]
287       have hfact2 : (n !).factorization p = 2 := by
288         have hIco : Finset.Ico 1 2 = ({1} : Finset ℕ) := by decide
289         calc (n !).factorization p = ∑ i ∈ Finset.Ico 1 2, n / p ^ i :=
290               Nat.factorization_factorial hp hlog2
291           _ = ∑ i ∈ ({1} : Finset ℕ), n / p ^ i := by rw [hIco]
292           _ = n / p ^ 1 := Finset.sum_singleton _ _
293           _ = n / p := by rw [pow_one]
294           _ = 2 := hd1
295       rw [hfact2] at hodd
296       obtain ⟨k, hk⟩ := hodd
297       omega
298   · -- ⟸ direction: for n ≤ 5, give the explicit square root pointwise
      (x = 1, 1, 2, 6, 12, 60)
299     intro h
300     obtain ⟨h0, h1, h2, h3, h4, h5⟩ := witnesses_zero_to_five
301     interval_cases n
302     · exact (isSquare_iff_exists_sq _).mpr ⟨1, h0.symm⟩
303     · exact (isSquare_iff_exists_sq _).mpr ⟨1, h1.symm⟩
304     · exact (isSquare_iff_exists_sq _).mpr ⟨2, h2.symm⟩
305     · exact (isSquare_iff_exists_sq _).mpr ⟨6, h3.symm⟩
306     · exact (isSquare_iff_exists_sq _).mpr ⟨12, h4.symm⟩
307     · exact (isSquare_iff_exists_sq _).mpr ⟨60, h5.symm⟩
308
309 /-- **Main theorem (existence-witness form, delivered alongside the iff
      form)**:
310     the Diophantine equation `x^2 = n! * n#` has a solution ⟺ `n ∈
      {0,1,2,3,4,5}`. -/
311 theorem exists_sq_factorial_mul_primorial_iff (n : ℕ) :
312     (∃ x : ℕ, x ^ 2 = n ! * primorial n) ↔ n ∈ ({0, 1, 2, 3, 4, 5} :
      Finset ℕ) := by
313   have hbridge : (∃ x : ℕ, x ^ 2 = n ! * primorial n) ↔ IsSquare (n ! *
      primorial n) := by
314     rw [isSquare_iff_exists_sq]
315     constructor
316     · rintro ⟨x, hx⟩
317       exact ⟨x, hx.symm⟩
318     · rintro ⟨r, hr⟩
319       exact ⟨r, hr.symm⟩
320   rw [hbridge, isSquare_factorial_mul_primorial_iff]
321   simp only [Finset.mem_insert, Finset.mem_singleton]
322   constructor <;> intro h <;> omega
323
324 end NFactPrimorialSquare
```

5.6 A refuted route, kept on the record

The route originally envisaged for Lemma 5 was a black-box splice of Bertrand's postulate (mathlib's `Nat.exists_prime_lt_and_le_two_mul`: a prime in $(m, 2m]$) across two dyadic sub-intervals. During the task this route was not merely found difficult but *strictly refuted as a black-box argument*: the set $\mathbb{P} \setminus \{13\}$ of all primes with 13 removed still has the Bertrand property — every interval $(m, 2m]$ contains one of its members, verified numerically for all $m \leq 200000$ and closed by hand on $m \in [7, 12]$ — yet for $n = 33$ the target window $(11, 16]$ is empty in that set, since with 13 removed none of $12, 14, 15, 16$ is a member. The window ratio $3/2$ is strictly below the Bertrand ratio 2, so no property-level argument at ratio 2 can close it; the countermodel is a statement about the proof method, not about the lemma (which is true, and is proved here by the chain-and-tail route). We record this as an honest negative result: the originally planned route was falsified, the falsification is retained in the record, and the shipped proof does not use it.

6 Verification evidence

- **Toolchain.** Lean `v4.34.0` (`leanprover/lean4:v4.34.0`), mathlib4 `v4.34.0`, revision `5ed29652`, pinned in the artifact (`proofs/lean-toolchain`).
- **Statement integrity.** The frozen snapshot `nfact-primorial-square.statement.txt` has sha256

  `0b13d08d6d915c2ebb25a070461a9d2a`
  `7d0364b12b3627c1b1efba5cc9f8ea48`,

  the statement source `nfact-primorial-square.lean` has sha256

  `638064201e96a3bdb42e75b00020c16e`
  `2fb27c2738c58fa3d4923cde2a451a8f`,

  and the final proof file `final-nfact-primorial-square.lean` (1094 lines, LF) has sha256

  `276cb2e698f6d201752eed3cb1104254`
  `d976f1a392ba3860a194eaa66986070a`.

The final audit compared all eight statement headers of the proof file against the frozen snapshot under two whitespace conventions (eight of eight match) and re-checked the kernel goal states against the snapshot's `#check` record (eight of eight match). The listing excerpts in this paper are byte-identical to the shipped proof files (independent mechanical record: `audit/listing-verify.txt`). The three `proofs/` sources are comment-cleaned editions (English docstrings): their code regions are byte-identical to the frozen files listed above (correspondence: `audit/comment-cleanup-diff.md`), and the post-cleanup kernel re-verification is recorded in `audit/kernel-rerun.txt`.

- **Well-definedness and non-degeneracy.** At the well-definedness gate the statements were compiled with placeholder bodies and probed with bare tactics: all nine probes fail, i.e. no single bare tactic discharges the frozen statements (the gate output ships verbatim as `audit/welldef-nfact-primorial-square.txt`). This is evidence about the statements, not a proof of them.

- **Kernel check.** A strict compile of the final file exits 0 with no `sorry` and no `admit` (whole-file scan including comments: zero hits; likewise zero hits for `axiom`, `unsafe`, `opaque`, `native_decide`, `implemented_by`). Three diagnostics remain, all linter/deprecation-class and non-rejectable under the audit discipline applied: an elaborator exponentiation-threshold hint on `18^100000` (intentional; the kernel checks the proof term independently), a deprecated `push_neg` alias, and one unused-variable linter note. The axiom printout below is the verbatim output of an independent re-run performed by the paper's packaging track on 2026-10-06 (strict compile exit 0; the wrapped fourth entry is terminal formatting; record: `audit/axioms.out`):

```
'NFactPrimorialSquare.isSquare_factorial_mul_primorial_iff' depends
     on axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.exists_sq_factorial_mul_primorial_iff'
    depends on axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.isSquare_iff_all_factorization_odd' depends
    on axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.exists_prime_between_div_three_and_div_two'
    depends on axioms: [propext,
 Classical.choice,
 Quot.sound]
'NFactPrimorialSquare.witnesses_zero_to_five' depends on axioms: [
    propext, Classical.choice, Quot.sound]
```

```
8 'NFactPrimorialSquare.primorial_factorization_eq_one' depends on
      axioms: [propext, Classical.choice, Quot.sound]
9 'NFactPrimorialSquare.primorial_factorization_eq_zero' depends on
      axioms: [propext, Classical.choice, Quot.sound]
10 'NFactPrimorialSquare.factorization_factorial_mul_primorial'
      depends on axioms: [propext, Classical.choice, Quot.sound]
```

Every entry is exactly the whitelist {propext, `Quot.sound`, `Classical.choice`}; `sorryAx` does not occur.

- **Grading.** The independent final audit's adjudication (2026-10-06, four checks all passed) grades the eight frozen statements: *complete proof* — zero `sorry`, axiom set inside the whitelist, statement text unchanged from the frozen snapshot, strict compile exit 0. A cleaned copy of the adjudication ships as `audit/gate3-final-adjudication.txt` (translated to English; machine-local path prefixes replaced by `<REPO-ROOT>` and dated task paths by `<source-task-id>`; verdict text and data unchanged — normalization disclosed in `audit/package-notes.md`).

7 Related work and novelty evidence

The source problem and its neighbours. The equation is one of two cases recorded as open in Novaković's paper [3] (quotation in Section 1); that paper studies $P(x) = \prod_{i=1}^{r} H_{n_i}$, a generalization of the Brocard–Ramanujan equation. The Brocard-side computational bound (10^9, attributed in the source task's records to Berndt and Galway) is background only: the exclusion of $n \geq 22$ in this paper is a pure theorem (Lemma 5) and does not rely on it; that reference is recorded as undecided at the full-text level in the source task and is not load-bearing here. The same author's companion paper [4] is recorded with the restricted wording that its object of study does not contain $n\#$.

Delimitations (wording obligations of the source task, discharged here one by one). (i) OEIS A055204 [9] asks for the *minimal* a making $n! \cdot a$ a square; the present theorem asks the *specific* value $a = n\#$ and is not an instance of A055204. (ii) Erdős problem #399 [8] concerns $n! = x^k \pm y^k$ — a different equation from $x^2 = n! \cdot n\#$ — and is marked DISPROVED (2026-02-04, counterexample $10! = 48^4 - 36^4$); it is recorded here only as same-family adjacency, never as an open anchor. (iii) Peixoto [5] (§5.3.1)

treats the primorial-core Pell equation $X^2 - DZ^2 = 1$ — an adjacent object that does not characterize which n make $n! \cdot n\#$ a square. (iv) Yudin [6] (square products of factorials and a conjecture of Erdős–Graham) is a different object; no collision, and the citation duty is discharged here. (v) Nagura [7] (for $x \geq 25$ there is a prime in $(x, 6x/5]$) and OEIS A166968 [10] (whose $a(2) = 8$ says: for $m \geq 8$ there is a prime in $(m, 3m/2]$) provide a literature-level shortest path toward Lemma 5 in the m-parametrization; the kernel proof shipped here does not go through it. Nagura's original text was not obtained; its statement is corroborated by several OEIS entries, and the load-bearing proof here is independent of it. (vi) The failure set of the interval property below $n = 22$ is $\{2, 3, 9, 21\}$ under the domain convention $n \geq 2$ ($n = 0, 1$ also fail, on an empty interval; this is irrelevant to Lemma 5, which starts at $n = 22$); the failure set was confirmed by three independent numerical recomputations.

Novelty evidence. An exclusivity review adjudicated: **no occupying record found**; search-strength label **strong**, with closing dates 2026-10-05 (deep-dig layer) and 2026-10-06 (an independent review layer that recompiled the load-bearing kernels, recomputed the numeric evidence from scratch and maintained every load-bearing claim). A zero hit is not novelty; the strongest statement licensed here is that no record of this object was found in the channels queried. The full query log, including every zero-hit query, ships as `audit/c9-record.md` in the deposit (English cleaned edition; conclusions and query data unedited). No "first formalization" statement is made anywhere in this paper.

channel	queries	result
literature (author self-report)	arXiv:2601.16757 body text	the case is on record as open (quotation in Section 1) [3]
arXiv metadata	primorial ∧ factorial ∧ perfect square; primorial ∧ factorial ∧ square; double factorial ∧ least common multiple; squarefree part ∧ factorial	zero hits each
arXiv metadata	Brocard ∧ primorial	one hit [5]: adjacent object (primorial-core Pell), does not characterize this equation
OpenAlex full text	brocard/primorial/square/factorial family; k-free Brocard family	nearest hits include [3, 4]; no same-form statement
OEIS keyword	"n! primorial square"; "factorial times primorial is a square"	hits exist; no same-form
OEIS keyword	"double factorial lcm square"; "Novakovic factorial"	zero hits
public Lean ecosystem (formal-conjectures tarball, content level)	`primorial`; `doubleFactorial`	occurrences in unrelated files only; no item in this domain
OEIS value columns (review layer)	x: 1,1,2,6,12,60; x^2: 1,4,36,144,3600	no registration of these objects
local mathlib (pinned v4.34.0), positive control	content scan with live control	channel live; no same-form item

8 Limitations and threats to validity

(a) **What is proved.** The kernel layer proves exactly the eight frozen statements of Listing 1; the grade of each is *complete proof*. Nothing beyond is claimed: no growth statement, no other multiplier in place of $n\#$, and no second equation.

(b) **The second case is not claimed.** The companion equation $x^2 = n!! \cdot [1, \ldots, n]$ of the same remark in [3] is not claimed: only an exclusion path for even n was obtained, and it is recorded as a follow-up item.

(c) **No registration claim.** No OEIS registration is claimed or performed; the review layer found no registration of the value columns, and any registration action is the author's own.

(d) **Independence from computational bounds.** The exclusion of $n \geq 22$ is a pure theorem and does not rely on the Brocard-side computational bound (10^9), which is full-text-undecided background and not load-bearing.

(e) **A stronger truth than the statement.** The hypothesis $n \geq 1$ of Lemma 4 is not used by its proof (the criterion holds at $n = 0$ as well); the lemma is stated as frozen, and we report the fact rather than silently strengthening the statement.

(f) **Comment-level deltas.** Three docstrings in the final file each carry one added explanatory sentence relative to the frozen snapshot (a notation note, the natural-division reading, and the correspondence to the informal statement); docstrings are comments and do not enter the statement comparison, which matched under both whitespace conventions.

(g) **Semantic fidelity of the formalization.** The theorems are about the Lean definitions. Their fidelity to the informal reading rests on the frozen statements being the intended encoding; a semantic round-trip check (formal statement back to natural language, adjudicated consistent) was performed at the formalization stage, and a further spot check of this paper's informal exposition against the frozen statements was done at assembly. A reader who disagrees with the encoding is disagreeing with the statement, not with the proof.

(h) **Nagura's original text.** Not obtained; the statement is used only as corroborated background (Section 7(v)) and is not load-bearing.

(i) **Novelty review limits.** The zero-hit verdict is a statement about the channels reached (arXiv metadata, OpenAlex full text, OEIS keyword and value faces, the public Lean ecosystem via a repository tarball, the local mathlib tree), not about the literature as a whole; a list of background items recorded as undecided at the full-text level is enumerated in the source task card (which stays in the source repository and is not shipped), and none of them is load-bearing for this result. No "first formalization" claim is made.

(j) **Numeric evidence is layered.** The failure set, the chain coverage and the A166968 correspondence are numeric- or literature-level evidence (the recomputation counts are stated where used); every theorem-level conclusion is adjudicated by the kernel and depends on no numeric evidence.

Data availability

The artifact deposited with this paper contains: the paper source and its PDF (`paper/`); the Lean sources (`proofs/`: the frozen statement snapshot, the statement source, the final proof file and the toolchain pin); the audit evidence (`audit/`: the final adjudication, the well-definedness gate output, the axiom printout, the listing verification record, the bibliography verification record, the full novelty-search record, the semantic fidelity

record, and the package self-description); and the metadata. Reproduction: with the pinned toolchain and mathlib revision, a strict compile of the deposited proof file exits 0, and appending the eight `#print axioms` lines reproduces the printout of Section 6. The companion claim of record for the same kernel result is deposited separately [11] (concept DOI `10.5281/zenodo.23177003`). This paper's own deposit is published on Zenodo (DOI `10.5281/zenodo.23179326`); the version history is recorded in the deposit's metadata README. The upload steps themselves are the author's (`UPLOAD.md` in the source repository lists them; it is not part of the deposit). Paper under CC BY 4.0, code under MIT.

Use of generative AI

The formalization and proofs reported in this paper were developed with AI assistance (large language model: `kimi-k3`) under human direction and review. All statements and proofs reported here were checked by the Lean 4 kernel (Lean v4.34.0, mathlib v4.34.0): every proof compiles with no `sorry`, and `#print axioms` reports only subsets of {`propext`, `Quot.sound`, `Classical.choice`}. The author reviewed all content and takes full responsibility for it.

Author contributions

The author selected and adjudicated the topic, verified the semantics of the formal statements against intent, signed off the verification gates, and made the submission decision. The AI system is not an author.

References

[1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. `https://doi.org/10.1007/978-3-030-79876-5_37`

[2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. `https://doi.org/10.1145/3372885.3373824`

[3] S. Novaković. The Diophantine equation $P(x) = \prod_{i=1}^{r} H_{n_i}$. arXiv:2601.16757, 2026. `https://arxiv.org/abs/2601.16757`

[4] S. Novaković. On a generalization of the Brocard–Ramanujan Diophantine equation. arXiv:2602.09687, 2026. https://arxiv.org/abs/2602.09687

[5] L. V. Peixoto. The Brocard problem: structural invariants, p-adic density, and the generative sieve. arXiv:2606.23485, 2026. https://arxiv.org/abs/2606.23485

[6] F. Yudin. Square products of factorials and a conjecture of Erdős and Graham. arXiv:2610.01899, 2026. https://arxiv.org/abs/2610.01899

[7] J. Nagura. On the interval containing at least one prime number. *Proceedings of the Japan Academy* 28 (1952), 177–181. https://doi.org/10.3792/pja/1195570997

[8] Erdős Problems website. Problem #399 ($n! = x^k \pm y^k$; marked DISPROVED, counterexample $10! = 48^4 - 36^4$). https://www.erdosproblems.com/399

[9] OEIS Foundation. Squarefree part of $n!$: $n!$ divided by its largest square divisor (sequence A055204). *The On-Line Encyclopedia of Integer Sequences*. https://oeis.org/A055204

[10] OEIS Foundation. Minimum k such that for all $m \geq k$ there is a prime p with $m < p < m(n+1)/n$ (sequence A166968). *The On-Line Encyclopedia of Integer Sequences*. https://oeis.org/A166968

[11] Aurora0134. Priority claim: complete solution set of $x^2 = n! \cdot n\#$ (exactly $n \in \{0, \ldots, 5\}$) — Lean 4 formalized proof and claim note, v1.0.2. *Zenodo*, 2026. Concept DOI: https://doi.org/10.5281/zenodo.23177003